

Starting a Digital Humanities Project: epiLog

Cite as: Martin Paul Eve, "Starting a Digital Humanities Project: epiLog", *eve.gd: Martin Paul Eve*, April 11, 2012, <https://doi.org/10.59348/pg8jt-da988>.

As part of a transparent development process, I wanted to announce that I'm starting, thanks to some funding and support from a colleague at Sussex, a Digital Humanities project that focuses upon object annotation and cross-medium comparison. The project is called epiLog and will be available during development [on my github](#).

The Concept

I constantly make annotations on books, films, photographs and many other objects. My typical process is to break down the object into manageable chunks and then write about these subsets. The way I write varies enormously. If it's public facing, my annotations will be narrative in style. If private, more like notes designed to jog my memory. What I thought would be really interesting, though, would be to provide people a platform for easily sharing, layering and comparing annotations. This comparison should happen, as I see it, not just on a single object (eg. we both used the same terms to describe a chapter of a book), but to cross-correlate between objects of different types. For instance, it is often said that Pynchon's *Gravity's Rainbow* is filmic in mode (cf. Brian McHale). Under epiLog, I hope it will be possible, when enough annotations are in the system, to find objects that, say, map onto one another, regardless of type (say, a David Lynch film and a Pynchon novel). In this way, we would be able to narrow the field to more likely matches for comparative studies, or at least for preliminary research purposes.

Day 1

I spent the first six hours of the project getting a Django environment up and running (I love python!) and integrating a [Twitter Bootstrap](#) environment and OpenID login, the latter taking the majority of the time. I have a homepage, a user profile (associated with an OpenID account) and avatar support ready to go.

I've never used Bootstrap before, but I have to say that it was a pleasurable experience. Although it tends towards homogenized styles, it really removes the hassle from styling. I can simply write well-formed HTML to a grid and have it displayed.

Component-wise: for those looking to integrate an OpenID login into Django, go no further than [django_openid_auth](#). Bear in mind that most providers fail if you run your development server on 127.0.0.1, so be sure to rebind with something like: `python ./manage.py runserver 192.168.0.2:8000`. If you want swanky login buttons, have a look at my modal dialog and Javascript snippet:

```

<!-- modal dialogs -->
<div class="modal fade" id="loginModal">
<div class="modal-header">
<a class="close" data-dismiss="modal">x</a>
<h3>Login to epiLog</h3>
</div>
<div class="modal-body">
<form action="" method="post">

<p>
epiLog uses OpenID for its authentication.
</p>
<p>
Username: <br/><input type="text" name="openid_username" id="openid_username" />
</p>
<p>Select your OpenID provider:</p>
<div class="btn-group" data-toggle="buttons-radio">
<a class="btn openurlbutton" data-toggle="button" id="aol"></a>
<a class="btn openurlbutton" data-toggle="button" id="blogger"></a>
<a class="btn openurlbutton" data-toggle="button" id="claimid"></a>
<a class="btn openurlbutton" data-toggle="button" id="flickr"></a>
<a class="btn openurlbutton" data-toggle="button" id="google"></a>
<a class="btn openurlbutton" data-toggle="button" id="livejournal"></a>
<a class="btn openurlbutton" data-toggle="button" id="myopenid"></a>
<a class="btn openurlbutton" data-toggle="button" id="technorati"></a>
<a class="btn openurlbutton" data-toggle="button" id="verisign"></a>
<a class="btn openurlbutton" data-toggle="button" id="vidoop"></a>
<a class="btn openurlbutton" data-toggle="button" id="yahoo"></a>
</div>

<a class="accordion-toggle" data-toggle="collapse" href="#collapseOne">
Advanced...
</a>

<div id="collapseOne" class="collapse">

```

```

<div class="accordion-inner">
  <p>Enter an OpenID URL:</p>

</div>
</div>

</p>
</form>
</div>
<div class="modal-footer">
  <a data-dismiss="modal" class="btn">Close</a> <a href="#" class="btn btn-primary"
  id="doLogin"><i class="icon-user icon-white"></i> Login</a>
</div>
</div>

```

```

<!-- Placed at the end of the document so the pages load faster -->
<script
src="http://ajax.googleapis.com/ajax/libs/jquery/1.7.1/jquery.min.js"></script>

```

```

<script type="text/javascript">

```

```

var provider = '';

```

```

$(document).ready(function() {

```

```

$(".openurlbutton").click(function(){
  provider = $(this).attr('id');
  setProvider(provider);
});

```

```

$("#openid_identifier").keyup(function(){
  setProvider(provider);
});

```

```

$("#doLogin").click(function(){
  $('form').submit();
});

```

```

});

```

```

  // a brief piece of script that will populate the openID identifier when the user
  clicks on a provider

```

```

  // it will also deal with the page aesthetics of clicking a provider

```

```

  function setProvider(provider)

```

```

  {

```

```

    providerBox = $('#id_openid_identifier');

```

```

    userName = $('#openid_username').val();

```

```
var selected = 0;

switch (provider)
{
    case "aol":
providerBox.val("http://openid.aol.com/" + userName);
break;
    case "blogger":
providerBox.val("http://" + userName + ".blogspot.com");
break;
    case "claimid":
providerBox.val("http://claimid.com/" + userName);
break;
    case "flickr":
providerBox.val("http://flickr.com/" + userName);
break;
    case "google":
providerBox.val("https://www.google.com/accounts/o8/id");
break;
    case "livejournal":
providerBox.val("http://" + userName + ".livejournal.com");
break;
    case "myopenid":
providerBox.val("http://" + userName + ".myopenid.com");
break;
    case "technorati":
providerBox.val("http://technorati.com/people/technorati/" + userName);
break;
    case "verisign":
providerBox.val("http://" + userName + ".pip.verisignlabs.com");
break;
    case "vidoop":
providerBox.val("http://" + userName + ".myvidoop.com");
break;
    case "yahoo":
providerBox.val("https://me.yahoo.com/");
break;
}

}

</script>
```

Input

I am extremely open to feedback, ideas/directions from interested parties. At present, I'm just going with the "build something cool" ethos, but if others have inside info on what I should change, API support, existing packages that I should know about, then I'd love to hear.